

PORTADA

Análisis y Diseño de Sistemas II

Semana 11 — Modelado UML Dinámico: Diagramas de Secuencia y Colaboración de Diseño

Universidad de Antioquia · Ingeniería de Sistemas



Objetos colaborando en
el tiempo

TRANSICIÓN

De "quién es responsable" a "quién le habla a quién"

La semana pasada cerramos los **nueve patrones GRASP** (Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection, Protected Variations): un vocabulario para decidir **qué clase debe tener cada responsabilidad** dentro del diseño.

Esa decisión responde una pregunta estática. Hoy respondemos la pregunta que sigue, dinámica: una vez repartidas las responsabilidades, **¿en qué orden colaboran esos objetos, en el tiempo, para cumplir un caso de uso completo?**

APERTURA

**Un diagrama de clases dice qué existe.
Ninguno dice, por sí solo, qué pasa primero.**

CONTRAEJEMPLO · RECORDANDO LA SEMANA 4

Lo que la caja negra dejaba oculto

En la semana 4 vimos el **Diagrama de Secuencia del Sistema (DSS)**: el sistema completo tratado como una sola caja negra, un único objeto `:Sistema` que recibe un evento del actor y devuelve una respuesta. Usamos como ejemplo el caso de uso **"Confirmar Pedido"**: el actor Cliente envía `confirmarPedido(idCliente)` y el sistema responde con `numeroConfirmacion`.

El DSS nunca dijo **cómo** el sistema calculó el total, ni **quién** generó el número de confirmación — eso quedaba deliberadamente oculto, "responsabilidad del diseño interno, no de ese diagrama". Hoy abrimos esa caja.

CONCEPTO

Qué es un diagrama de secuencia de diseño

Un **diagrama de secuencia de diseño** muestra los **objetos internos** del sistema (instancias de las clases del diseño) y los **mensajes** que se envían entre sí, ordenados en el tiempo, para cumplir un escenario concreto de un caso de uso.

A diferencia del DSS de la semana 4 —que trataba el sistema como una única caja negra :Sistema — aquí **cada objeto de diseño aparece por separado**: se ve exactamente quién le envía el mensaje a quién, y en qué orden ocurre cada paso dentro del sistema.

Línea de vida y activación

- **Línea de vida (lifeline):** línea vertical punteada que representa la existencia de un objeto a lo largo del tiempo. Se etiqueta `:NombreClase` (objeto anónimo) o `nombre:NombreClase` (objeto con nombre propio). El tiempo avanza de arriba hacia abajo, igual que en el DSS.
- **Activación (activation / execution occurrence):** una barra rectangular delgada sobre la línea de vida que marca el período en que ese objeto está **ejecutando** un método — desde que recibe un mensaje hasta que termina de procesarlo (incluye el tiempo de cualquier llamada que él mismo haga a otros objetos).

 Líneas de vida en el tiempo

CONCEPTO · NOTACIÓN

Mensajes: síncrono, asíncrono y de retorno

- **Mensaje síncrono:** flecha sólida con punta llena. El objeto que envía el mensaje **se detiene y espera** a que el receptor termine de procesarlo antes de continuar — es el caso más común entre objetos dentro de una misma aplicación.
- **Mensaje asíncrono:** flecha sólida con punta abierta (en forma de "v"). El emisor **continúa su propia ejecución** sin esperar respuesta inmediata — típico de colas de mensajes, eventos o notificaciones que se disparan y no bloquean.
- **Mensaje de retorno:** flecha punteada que regresa del receptor hacia el emisor, con el valor devuelto al terminar la activación. Es el mismo concepto de "retorno" que ya vimos en el DSS de la semana 4, ahora entre objetos internos.

CONCEPTO · NOTACIÓN

Creación y destrucción de objetos

- **Mensaje de creación:** flecha que apunta directamente al **inicio** de una línea de vida nueva (no a un punto intermedio), normalmente etiquetada «create» o con el nombre del constructor. Indica que ese objeto no existía antes y se instancia en ese momento del escenario.
- **Mensaje de destrucción:** se marca con una **X** al final de la línea de vida del objeto destruido, indicando que deja de existir a partir de ese punto — útil en lenguajes con manejo explícito de memoria; en la práctica de este curso se usa poco, pero conviene reconocerlo.

Estos dos elementos conectan directamente con el patrón GRASP **Creator** visto la semana pasada: el diagrama de secuencia es donde se **ve** literalmente qué objeto crea a cuál.

CONCEPTO · NOTACIÓN

Marcos de interacción combinados (introducción)

Cuando el flujo de mensajes no es una línea recta, UML define **marcos de interacción combinados (combined fragments)**: un recuadro con una etiqueta que envuelve un grupo de mensajes y le da una condición.

- **alt (alternative)**: el recuadro se divide en secciones con una guarda ([condición]) cada una — solo se ejecuta la sección cuya condición es verdadera, como un if/else.
- **loop** : el conjunto de mensajes dentro del recuadro se repite mientras se cumpla la condición indicada, como un ciclo.

Hoy solo se introducen a nivel de reconocimiento — basta con identificarlos al leer un diagrama; construirlos con detalle no es el foco de esta sesión.

CONCEPTO · GRASP

El Controller como punto de entrada

El patrón GRASP **Controller** (semana 10) asigna la responsabilidad de recibir un evento de sistema a un objeto que **no es la interfaz de usuario** y coordina lo que sigue, sin hacer él mismo el trabajo de negocio.

En un diagrama de secuencia de diseño, el **Controller es casi siempre el primer objeto interno** que recibe el mensaje: el mismo evento de sistema que en el DSS de la semana 4 llegaba a :Sistema ahora llega, específicamente, a un objeto Controller — y desde ahí se reparte hacia los demás objetos según lo que decidió GRASP.

CONCEPTO

El diagrama de secuencia como verificación de GRASP

Asignar responsabilidades con GRASP es una decisión **de diseño**; el diagrama de secuencia es la forma de **comprobar** que esa asignación realmente funciona en la práctica, mensaje por mensaje:

- Si al trazar la secuencia un objeto necesita datos que no tiene y que nadie le pasa, probablemente la responsabilidad quedó mal asignada (viola Information Expert).
- Si el Controller termina haciendo cálculos de negocio él mismo en vez de delegarlos, el diagrama lo deja ver de inmediato.
- Si dos objetos terminan enviándose mensajes de ida y vuelta excesivos, probablemente hay un problema de acoplamiento (Low Coupling).

El diagrama de secuencia, entonces, no solo documenta el diseño: lo **pone a prueba** contra un caso de uso real.

EJEMPLO TRABAJADO · PARTE 1

Abriendo la caja de "Confirmar Pedido"

Retomamos el mismo caso de uso de la semana 4. En el DSS, el actor **Cliente** enviaba un único evento a la caja negra:

```
Cliente -> :Sistema : confirmarPedido(idCliente)
:Sistema --> Cliente : numeroConfirmacion
```

Hoy diseñamos qué hay **dentro** de :Sistema . Usamos tres objetos de diseño: :ControladorPedido (el **Controller**, punto de entrada), :Pedido (conoce sus líneas de producto y su total — **Information Expert**) y :Pago (un objeto nuevo que :ControladorPedido crea — aplicando **Creator**, porque el controlador es quien agrega y usa el pago).

EJEMPLO TRABAJADO · PARTE 2

La secuencia completa, mensaje por mensaje

```
1. Cliente -> :ControladorPedido : confirmarPedido(idCliente)
2. :ControladorPedido -> :Pedido : calcularTotal()
3. :Pedido --> :ControladorPedido : total
4. :ControladorPedido -> :Pago : «create»(idPedido, total)
5. :ControladorPedido -> :Pago : confirmar()
6. :Pago --> :ControladorPedido : numeroConfirmacion
7. :ControladorPedido --> Cliente : numeroConfirmacion
```

El mensaje 1 es exactamente el evento de sistema que ya conocíamos del DSS. Los mensajes 2 a 6 son **nuevos**: son la colaboración interna que el DSS mantenía oculta. El mensaje 7 es el mismo retorno que el DSS ya mostraba, ahora con su origen interno explícito.

Leyendo la secuencia con lentes de GRASP

Objeto	Rol en la secuencia	Patrón GRASP
:ControladorPedido	Recibe el evento (mensaje 1), delega el cálculo, crea :Pago , le pide confirmar y devuelve el resultado	Controller
:Pedido	Conoce sus propias líneas de producto y calcula su total (mensaje 2–3)	Information Expert
:Pago	Se crea a partir de datos que ya existen (mensaje 4) y encapsula la lógica de confirmación (mensaje 5–6)	Creator

Ningún objeto conoce más de lo que necesita para su propia tarea: :Pedido no sabe nada de pagos, :Pago no sabe nada del cliente. Esa separación es exactamente lo que **Low Coupling** y **High Cohesion** buscaban en el diseño de clases — el diagrama de secuencia confirma que se cumple en tiempo de ejecución.

CONTRA EJEMPLO

Errores comunes al trazar la secuencia

- **Saltarse el Controller:** hacer que el actor le envíe el evento directamente a un objeto de negocio (por ejemplo, a :Pedido) en vez de a un Controller — mezcla la interfaz externa con la lógica interna.
- **Un objeto "todopoderoso":** si un solo objeto recibe casi todos los mensajes y hace casi todo el trabajo, probablemente no se aplicaron bien los patrones GRASP de la semana pasada — el diagrama de secuencia es donde ese problema se hace visible.
- **Confundir mensaje con llamada de método sin más:** cada flecha debe corresponder a un método que existirá en el diagrama de clases de diseño — no se inventan mensajes que después no tienen dónde vivir.

 Errores comunes en el
diseño de la secuencia

TENSIÓN CONCEPTUAL

Misma información, dos formas de organizarla

El diagrama de secuencia organiza los mensajes **en el tiempo** (de arriba hacia abajo). Existe una notación alternativa, el **diagrama de colaboración** (llamado *diagrama de comunicación* en UML 2), que muestra **los mismos objetos y los mismos mensajes**, pero organizados **en el espacio**: los objetos se dibujan como nodos conectados por líneas, y el orden temporal se indica con **números de secuencia** sobre cada mensaje en vez de con la posición vertical.

Ninguna de las dos notaciones "vale más" que la otra — son la misma colaboración vista con un énfasis distinto.

Diagrama de colaboración: la misma escena, en el espacio

En un diagrama de colaboración, cada objeto es un nodo (igual que en un diagrama de clases: `:ControladorPedido` , `:Pedido` , `:Pago`) unido por líneas a los objetos con los que se comunica. Sobre cada línea se dibuja una flecha con un **número de secuencia** que indica en qué orden ocurre ese mensaje:

```
Cliente ---1: confirmarPedido(idCliente)---> :ControladorPedido
:ControladorPedido ---2: calcularTotal()---> :Pedido
:ControladorPedido ---3: «create»(total)---> :Pago
:ControladorPedido ---4: confirmar()---> :Pago
```

Los mensajes anidados dentro de otro (por ejemplo, algo que `:Pago` hiciera como parte de responder al mensaje 4) se numerarían `4.1` , `4.2` , siguiendo la misma lógica de anidamiento.

 Objetos conectados
espacialmente

CONCEPTO

Cuándo preferir cada notación

- **Diagrama de secuencia:** se prefiere cuando lo importante es comunicar **el orden exacto y la duración relativa** de las interacciones — por ejemplo, para revisar con el equipo si un flujo tiene demasiados pasos o esperas innecesarias. Es la notación más usada en la práctica profesional y en este curso.
- **Diagrama de colaboración:** se prefiere cuando lo importante es mostrar **la red de relaciones entre objetos** — quién conoce a quién — de forma compacta, sin ocupar tanto espacio vertical como una secuencia larga. Es útil como complemento visual rápido, especialmente con pocos objetos.

Ambas se derivan del mismo análisis: primero se piensa la colaboración (con ayuda de GRASP), y después se elige cómo dibujarla según qué se quiera resaltar.

 Dos notaciones, una misma colaboración

INTERACCIÓN

Tracen la secuencia de su propio caso de uso

En el chat, sobre un caso de uso simple de su proyecto (o retomando el DSS que ya construyeron en la semana 4):

1. Identifiquen qué objeto de su diseño actuaría como **Controller** — el que recibe el evento de sistema.
2. Listen, en texto, de 3 a 5 mensajes que ese Controller enviaría a otros objetos para cumplir el caso de uso (formato `objeto1 -> objeto2 : mensaje()`).
3. Para cada mensaje, digan qué patrón GRASP justifica que ese objeto —y no otro— lo reciba.

10 minutos · respondan en el chat

SÍNTESIS

Lo que hay que llevarse de hoy

1. El diagrama de secuencia de diseño abre la caja negra del DSS: muestra los **objetos internos** colaborando, no solo actor y sistema.
2. Notación clave: **línea de vida, activación, mensaje síncrono/asíncrono, retorno, creación y destrucción** de objetos, y marcos combinados (`alt` , `loop`) para flujos condicionales o repetitivos.
3. El **Controller** (GRASP) es casi siempre el punto de entrada del diagrama; los demás mensajes reflejan cómo se repartieron las demás responsabilidades.
4. El diagrama de secuencia no solo documenta el diseño: lo **verifica** contra un caso de uso real, mensaje por mensaje.
5. El **diagrama de colaboración** muestra la misma información en el espacio, con números de secuencia — alternativa complementaria, no una notación distinta en el fondo.

 Ideas clave de la sesión

CIERRE

Lo que sigue esta semana

El **miércoles** se ve el **diagrama de clases de diseño**: ahí van a aparecer, formalmente, los métodos que hoy escribimos como mensajes en la secuencia (`calcularTotal()` , `confirmar()` , etc.) — el diagrama de secuencia de hoy es, en buena medida, el borrador de esos métodos.

El **viernes** hay **laboratorio de aplicación de GRASP** sobre el proyecto propio de cada equipo.

 Próximas sesiones de la semana